House prices | Salary prediction | Sales forecasting | Trend analysis

📊 LINEAR REGRESSION

Predicting continuous outcomes using a best-fit straight line

💡 1. Definition & Intuition

A supervised learning algorithm used to model the relationship between a dependent variable (target) and one or more independent variables (features) using a straight line.

Input → Feature Scaling → Fit Line → Predict → Compute MSE

🧮 2. Mathematical Equations

$$y = w_1x_1 + ... + w_nx_n + b$$

Loss (MSE): $1/n \, \Sigma \, (y_i - \hat{y}_i)^2$

y = predicted output
x_i = input feat.
w_i = weights
b = bias (intercept)

📐 3. Geometrical Interpretation

Creates a best-fit straight line in 2D, a plane in 3D, and a hyperplane in higher dimensions. The goal is to minimize the vertical distance (error) between data points and the line. (↙ error ± reality.)



* (blue points): 'reality'. Each dot instance/ λ (dots con q entrenaste modelo)
* (red line): es tu modelo. Regla marem. y tu algorit. creó para "entrenar" tend. los puntos
* ?: 5 years working. Salary ?

🎯 4. When to Use & Pros/Cons

When to Use:

- Output is continuous (numeric).
- Relationship between variables is approximately linear.
- Forecasting and trend analysis (e.g., House prices, Salary, Sales).

Advantages:

- ✓ Simple and easy to implement
- ✓ Fast training
- ✓ Interpretable results

Disadvantages:

- ✗ Poor performance on non-linear data
- ✗ Sensitive to outliers
- ✗ Can underfit complex datasets

💻 5. Python Implementation

```
# Sample data
X = [[1], [2], [3], [4], [5]]
y = [2, 4, 6, 8, 10]

# Split data (training and testing split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model (create model and train)
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction (predict on test data)
y_pred = model.predict(X_test)

# Evaluation (evaluate the model)
print('MSE:', mean_squared_error(y_test, y_pred))
print('Slope:', model.coef_)
print('Intercept:', model.intercept_)
```

⚙️ 6. Assumptions & Hyperparameters

Assumptions:

- Linear relationship between X and y
- No multicollinearity (input features X too similar)
- Homoscedasticity (constant variance of errors) ~> error for predicting the salary for junior vs CEO. (we don't want this)
- Independence of observations
- Normally distributed residuals (♡ small errors)

Key Hyperparameters:

- **fit_intercept**: whether to calculate intercept (where the line starts)
- **copy_X**: copy or overwrite data (if u set to True → 'protects orig. data')
- **n_jobs**: parallel computation (if applicable)

⚠️ WARNING / PITFALLS

⚠️ Very sensitive to extreme outliers, which can drastically pull the regression line away from the true trend.

💡 TIPS & TRICKS

DON'T FORGET

🔑 Always check residual plots to verify the homoscedasticity assumption. ~> PLOT error } error = $y_{true} - y_{predicted}$ (eje y) (eje x)

❓ PRACTICE QUESTIONS

🎯 Q: How do you handle violations of the linearity assumption in Linear Regression? Mathem. transf (log etc) / changing model that captures non-linear relations.

🔑 **LINEAR REGRESSION:** The foundational algorithm that fits a line to minimize squared prediction errors.

Spam detection | Disease prediction

📊 LOGISTIC REGRESSION

Binary classification mapping outputs to probabilities using the Sigmoid function

💡 1. Definition & Intuition

A supervised machine learning algorithm used for classification. It predicts the probability that an input belongs to a particular class, mapping values between 0 and 1.

Input → Linear Combination (z) → Sigmoid Activation (σ) → Probability → Threshold

🧮 2. Mathematical Equations

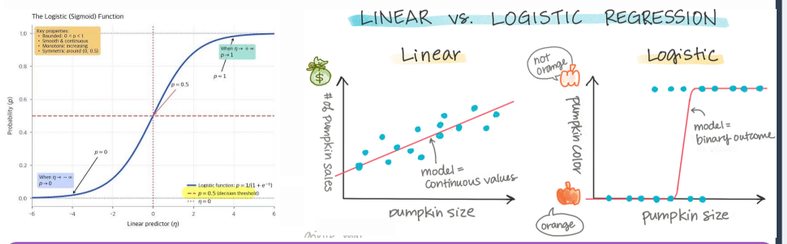
Linear: $z = w^T x + b$

Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$

Decision: ŷ = 1 if $\sigma(z) \geq 0.5$ else 0

📐 3. Geometrical Interpretation

Creates a linear decision boundary separating classes. Uses an S-shaped sigmoid curve to map the linear outputs into probabilities.



🎯 4. When to Use & Pros/Cons

When to Use:

- Output is categorical (binary).
- Need probability scores.
- Data is linearly separable (e.g., Spam detection, Disease prediction).

Advantages:

- ✓ Simple, fast, and easy to interpret
- ✓ Provides probability outputs
- ✓ Less prone to overfitting on simple data

Disadvantages:

- ✗ Cannot solve complex non-linear problems
- ✗ Assumes linear decision boundary

💻 5. Python Implementation

```
# Sample Data
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset (training & testing set)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model (Create model)
model = LogisticRegression()

# Train (Train the model)
model.fit(X_train, y_train)

# Prediction (Predict on test data)
y_pred = model.predict(X_test)

# Accuracy (Model accuracy)
print('Accuracy:', accuracy_score(y_test, y_pred))

# Probability (Probability of class [0, 1] for input 3.5)
print(model.predict_proba([[3.5]]))
```

⚙️ 6. Assumptions & Hyperparameters

Assumptions:

- Binary outcome variable
- Independent observations
- No multicollinearity
- Linear relationship between log-odds and features

Key Hyperparameters:

- **penalty**: regularization type (l1, l2) *
- **C**: regularization strength
- **solver**: optimization algorithm → lbfgs, liblinear, saga
- **max_iter**: maximum iterations
- **class_weight**: handle imbalanced data

* L1 (Lasso): + drástico. Multiplica x 0 y elim. feature

* L2 (Ridge): + suave. No elim. variable DEFAULT

⚠️ WARNING / PITFALLS

⚠️ Using an unregularized logistic regression on datasets with more features than samples will lead to severe overfitting.

💡 TIPS & TRICKS

🔑 The 'C' parameter is the inverse of regularization strength. Smaller C means stronger regularization.

❓ PRACTICE QUESTIONS

🎯 Q: Why do we use Sigmoid instead of a standard linear function for classification?

🔑 LOGISTIC REGRESSION: It's a regression algorithm in name, but a powerful linear classifier in practice!

Customer segmentation | Credit risk assessment | High interpretability

DECISION TREES

Non-parametric splitting of data into regions mimicking human decision-making

NOTE:
Divides the feature space into rectangular regions via orthogonal splits. Each split creates a decision boundary parallel to an axis.

1. Definition & Intuition

A supervised algorithm for classification and regression. It divides feature space into rectangular regions using a tree-like structure of internal nodes (tests) and leaf nodes (predictions).

Root Node → Evaluate Feature Splits → Maximize Info Gain → Branch → Leaf Node Prediction

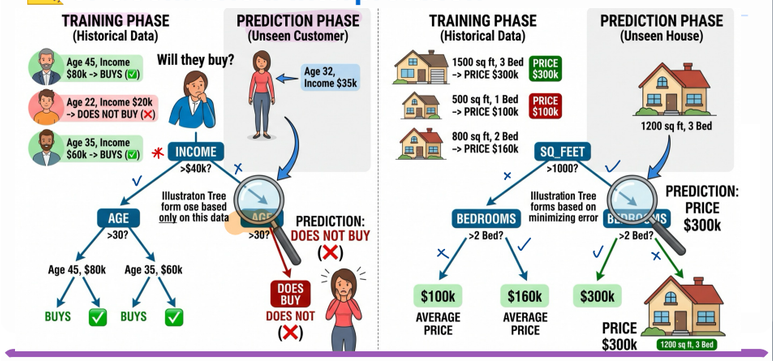
2. Mathematical Equations

Entropy: $- \Sigma p_i \log_2(p_i)$

Information Gain: $Entropy(S) - \Sigma (|S_v| / |S|) * Entropy(S_v)$

- Entropy measures caos/impurity → ↑ entropy desordan :(
- IG: reduct. impurity no IG = entropy antes - Entropy after
- Max IG ⇒ reduce entropy.

3. Geometrical Interpretation



4. When to Use & Pros/Cons

When to Use:

- Need highly interpretable models.
- Data contains non-linear relationships.
- Both numerical and categorical data exist.
- Feature importance is required.

Advantages:

- ✓ Easy to understand and visualize
- ✓ Handles non-linear data natively
- ✓ No feature scaling required

Disadvantages:

- ✗ High risk of overfitting (memorizes data)
- ✗ Unstable with small data changes
- ✗ Biased toward dominant classes

5. Python Implementation

```
# Sample Dataset
X = [[25, 40000], [35, 60000], [45, 80000], [20, 20000]]
y = [0, 1, 1, 0]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = DecisionTreeClassifier()

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- No assumptions about data distribution.
- Features should meaningfully split data.

Key Hyperparameters:

- **criterion**: gini or entropy (function to split)
- **max_depth**: maximum depth of the tree
- **min_samples_split**: minimum samples required to split an internal node
- **min_samples_leaf**: minimum samples required at a leaf node
- **max_features**: number of features to consider when looking for best split

⚠ WARNING / PITFALLS

⚠ An unconstrained Decision Tree will grow until every leaf is pure, resulting in massive overfitting.

TIPS & TRICKS

🔑 Always prune trees using max_depth or min_samples_leaf to guarantee generalization.

❓ PRACTICE QUESTIONS

🎯 Q: What is the difference between Gini Impurity and Entropy in node splitting?

DECISION TREES: Highly interpretable and powerful, but must be carefully pruned to survive unseen data!

Recommender systems | Pattern recognition

K-NEAREST NEIGHBORS (KNN)

Instance-based learning utilizing distance metrics and majority voting

1. Definition & Intuition

A non-parametric, instance-based algorithm that classifies a new data point based on the majority class (or average value) among its 'K' closest neighbors in the training set.

Store Training Data → Receive New Point → Calculate Distances → Find K Nearest → Vote/Average

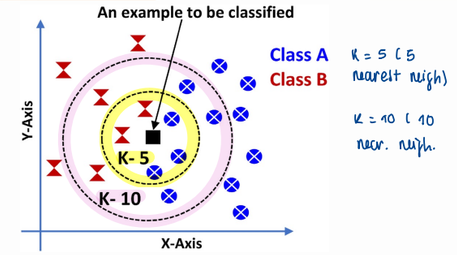
2. Mathematical Equations

Euclidean: $d(x,y) = \sqrt{(\Sigma(x_i - y_i)^2)}$

Manhattan: $d(x,y) = \Sigma |x_i - y_i|$

3. Geometrical Interpretation

Creates complex, non-linear decision boundaries based on distance. Small K yields jagged boundaries; large K creates smoother separations.



4. When to Use & Pros/Cons

When to Use:

- Dataset is small to medium-sized.
- Non-linear decision boundaries exist.
- Fast training is acceptable (training is instantaneous).

Advantages:

- ✓ Simple and intuitive
- ✓ No actual training phase (lazy learning)
- ✓ Naturally handles multi-class problems

Disadvantages:

- ✗ Very slow prediction time for large datasets
- ✗ Highly sensitive to unscaled features
- ✗ Memory intensive (stores all data)

5. Python Implementation

```
# Sample Dataset
X = [[2, 3], [3, 4], [1, 1], [8, 9], [9, 10], [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = KNeighborsClassifier(n_neighbors=3)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- Similar data points exist near each other.
- Data is properly scaled (crucial).
- Meaningful distance metric is chosen.

Key Hyperparameters:

- **n_neighbors**: number of neighbors
- **weights**: uniform or distance
- **metric**: distance metric
- **p**: power parameter for Minkowski distance
- **algorithm**: auto, ball_tree, kd_tree, brute

Algorithm	Summary (What it does)	When to use
auto	Selects the fastest method automatically.	Default (ideal choice).
brute	Brute force: checks against every point.	Very small datasets.
kd_tree	Spatial tree structure in rectangular boxes.	Few columns / features.
ball_tree	Spatial tree structure in concentric spheres.	Many dimensions / features.

⚠ WARNING / PITFALLS

⚠ Failing to scale numerical features will cause features with larger ranges to completely dominate the distance calculation.

TIPS & TRICKS

🔑 Use an odd number for 'K' in binary classification to prevent voting ties.

❓ PRACTICE QUESTIONS

🎯 Q: Why is KNN called a 'lazy learner'? → ⊗ training phase.

🔑 **K-NEAREST NEIGHBORS:** Tell me who your neighbors are, and I will tell you what class you belong to!

Fraud detection | Financial risk modeling | Disease prediction

RANDOM FOREST

Ensemble learning utilizing Bagging and Random Features to dominate complex data

1. Definition & Intuition

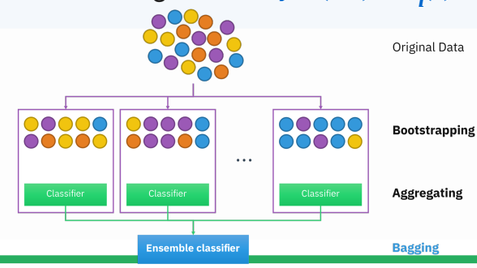
An ensemble algorithm that builds multiple independent decision trees on bootstrapped subsets of data and features, combining their outputs to improve accuracy and prevent overfitting.

Concept	Simple Meaning	How it Works	Key Effect
Bootstrap	Random data subsets with replacement.	Samples original rows randomly, allowing duplicates. ~ remaining never used OOB(valid.)	Gives each tree a unique perspective.
Bagging	Bootstrap Aggregating.	Combines bootstrap row sampling with model aggregation (average or vote).	Reduces overall variance, generalizes better, and prevents overfitting. ← 'each model learns indep.'
Random Feature Selection	Random subset of columns ($\sqrt{p}$).	Restricts node splits to random columns.	Decorrelates individual trees (combined with bagging).

2. Mathematical Equations

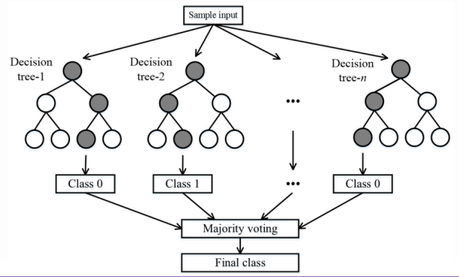
Classification (Voting): $\hat{y} = mode(T_1(x), ..., T_n(x))$

Regression: $\hat{y} = (1/n)\ \Sigma\ T_i(x)$ (average predict)



3. Geometrical Interpretation

Combines multiple jagged decision boundaries from individual trees to produce a much smoother and more generalized final decision surface.



4. When to Use & Pros/Cons

When to Use:

- Need very high accuracy and stability.
- Dataset contains complex, non-linear relationships.
- A standard decision tree is severely overfitting.

Advantages:

- ✓ Outstanding accuracy and generalization
- ✓ Naturally handles missing values well
- ✓ Provides built-in Feature Importance metrics

Disadvantages:

- ✗ Slower prediction times than a single tree
- ✗ High memory consumption for large forests
- ✗ Operates as a 'black box' (less interpretable)

5. Python Implementation

```
# Sample Dataset
X = [[25, 40000], [35, 60000], [45, 80000], [20, 20000],
[50, 90000]]
y = [0, 1, 1, 0, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = RandomForestClassifier(n_estimators=100,
random_state=42)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- Individual trees must be diverse/uncorrelated.
- No strict statistical distribution assumptions.

Key Hyperparameters:

- **n_estimators**: Number of trees in the forest
- **max_depth**: Maximum depth of the trees
- **min_samples_split**: Minimum samples required to split an internal node
- **min_samples_leaf**: Minimum samples required at a leaf node
- **max_features**: Number of features considered at each split
- **bootstrap**: Whether bootstrap sampling (with replacement) is used

⚠️ WARNING / PITFALLS

⚠️ While resistant to overfitting, exceptionally noisy datasets can still trick Random Forests into memorizing noise.

💡 TIPS & TRICKS

🔑 Start tuning with n_estimators and max_features; these yield the highest performance dividends.

❓ PRACTICE QUESTIONS

🎯 Q: How does Random Forest ensure the individual trees are decorrelated? (see table)
↳ Bagging + Rand. feature select at each split

🔑 **RANDOM FOREST:** The wisdom of the crowd applied to decision trees—diversity guarantees strength!

Text classification | Spam filtering | Sentiment analysis (NLP)

NAIVE BAYES

Probabilistic classification founded on Bayes' Theorem and feature independence

1. Definition & Intuition

A probabilistic classification algorithm based on Bayes' Theorem. It calculates probabilities for each class and assigns the highest probability class, assuming all features are strictly independent.

Calculate Priors → Calculate Likelihoods → Apply Bayes Theorem → Predict Maximum Posterior

2. Mathematical Equations

prob. spam knowing that containing word 'free' → prob of 'free' knowing spam (spam)

Posterior Prob | Likelihood | Prior | Evidence

Bayes: $P(C|X) = [P(X|C) * P(C)] / P(X)$

Naive Assumption: $P(C|x) \propto P(c) \prod P(x_i|c)$

X : feat.vector
C : class
P(X|C): prob of X, knowing C

Bayes → P(A|B) = P(B|A) P(A) / P(B)

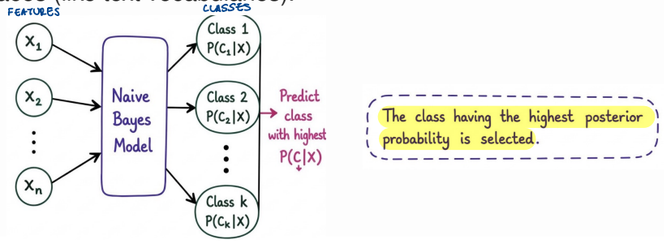
If (P | what u know)

DON'T FORGET

- **Training Phase**: You learn P(X|C) (the likelihood of features given the class) and P(c) (the class prior probability) from your training data.
- **Prediction Phase**: You use Bayes' theorem to calculate P(C|X) (the posterior probability of the class given the features) and pick the class with the highest probability.

3. Geometrical Interpretation

Relies on conditional probability densities rather than geometric margins. Works exceptionally well in massive high-dimensional spaces (like text vocabularies).



The class having the highest posterior probability is selected.

4. When to Use & Pros/Cons

When to Use:

- Dataset is high-dimensional (e.g., text/NLP).
- Fast prediction and training is mandated.
- Limited training data is available.

Advantages:

- ✓ Extremely fast and computationally lightweight
- ✓ Outstanding for text classification/spam filtering
- ✓ Performs well even with small datasets

Disadvantages:

- ✗ The 'Naive' independence assumption is rarely true in reality
- ✗ Suffers from the 'Zero-Frequency' probability problem

↳ Laplace

5. Python Implementation

```
# Sample Dataset
X = [[2, 3], [3, 4], [1, 1], [8, 9], [9, 10], [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- Features are strictly independent of each other.
- Data distribution matches the model type (Gaussian, Multinomial).

Key Hyperparameters:

- **var_smoothing**: stability for GaussianNB
- **alpha**: smoothing parameter
- **fit_prior**: whether to learn class priors
- **class_prior**: prior probabilities

⚠ WARNING / PITFALLS

⚠ If a categorical variable has a category in the test set not seen in training, it assigns a 0 probability (fix with Laplace smoothing).

TIPS & TRICKS

🔑 Use MultinomialNB for discrete counts (e.g., word frequencies) and GaussianNB for continuous features.

PRACTICE QUESTIONS

🎯 Q: Why is this algorithm called 'Naive'?

↳ Assum. indep ÷ variables → rarely true !

🔑 **NAIVE BAYES:** Despite its unrealistic independence assumption, it remains incredibly robust in practice!

Face detection | Image classification

SUPPORT VECTOR MACHINE (SVM)

Finding the optimal hyperplane that maximizes the geometric margin between classes

1. Definition & Intuition

A supervised model that finds the optimal decision boundary (hyperplane) that best separates data points into classes by maximizing the geometric margin between them.

Map Input Space → Find Support Vectors → Maximize Margin → Define Hyperplane Boundary

* SV : nearest data points to boundary.
* Margin: distance ∠ boundary SV

2. Mathematical Equations

Hyperplane: $w^T x + b = 0$

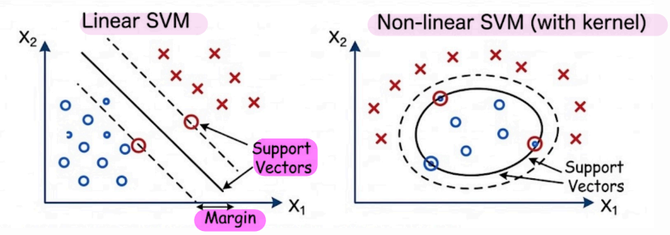
Margin Width: $2 / ||w||$

Prediction: $f(x) = sign(w^T x + b)$ (b: intercept)

(w: weights, x: input feature vector) → PREDICTS CLASS LABEL

3. Geometrical Interpretation

Finds the 'widest street' (margin) between the classes. Support vectors are the critical data points lying exactly on the edge of this margin.



4. When to Use & Pros/Cons

When to Use:

- High-dimensional data spaces (where dimensions > samples).
- Clear margin of separation exists.
- Complex non-linear boundaries are required (using Kernels).

Advantages:

- ✓ Highly effective in high-dimensional spaces
- ✓ Memory efficient (only uses support vectors)
- ✓ Extremely versatile via the 'Kernel Trick'

Disadvantages:

- ✗ Computationally slow for very large datasets
- ✗ Highly sensitive to noise and overlapping outliers
- ✗ Difficult to interpret mathematically

5. Python Implementation

```
# Sample Dataset
X = [[2, 3], [3, 4], [1, 1], [8, 9], [9, 10], [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = SVC(kernel='linear')

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- Data classes are separable (or soft-margin is used).
- Feature scaling is absolutely required.

Key Hyperparameters:

- **kernel**: linear, poly, rbf, sigmoid
- **C**: regularization parameter
- **gamma**: kernel coefficient
- **degree**: polynomial degree
- **probability**: probability estimates

⚠ WARNING / PITFALLS

⚠ A very high 'C' value forces a strict margin, easily causing overfitting to outliers. A low 'C' allows a softer, wider margin.

TIPS & TRICKS

🔑 The 'Kernel Trick' allows SVM to project data into higher dimensions without computing the coordinates explicitly.

PRACTICE QUESTIONS

🎯 Q: Explain the role of Support Vectors in defining the SVM model. → define hyperplane.

🔑 **SVM:** Maximizing the margin ensures the strongest possible boundary for generalization!

Email Spam Detection | E-commerce Sales Prediction | Credit Score Assessment

GRADIENT BOOSTING

Sequential ensemble learning optimizing errors via Gradient Descent

Healthcare Diagnosis | Stock Price Prediction | Fraud Detection

1. Definition & Intuition

An advanced ensemble learning algorithm that builds models sequentially. Each new weak learner (tree) is specifically trained to correct the residual errors made by the previous models.

Train Initial Model → Calculate Residuals → Train New Tree on Residuals → Add to Ensemble → Repeat

2. Mathematical Equations

Model Update: $F_M(x) = F_0(x) + \Sigma \gamma_m h_m(x)$

Goal: Minimize Loss Function $L(y, F_m(x))$

3. Geometrical Interpretation

Moves in the direction of steepest descent within the loss function space. Step-by-step, it sculpts an incredibly precise decision boundary.

[IMAGE/DIAGRAM: Initial Model → Model 1 (corrects error) → Model 2 (corrects error) → Optimal Final Model]

4. When to Use & Pros/Cons

When to Use:

- Highest predictive accuracy is the primary goal.
- Data has complex, nuanced tabular patterns.
- Sufficient time exists for hyperparameter tuning.

Advantages:

- ✓ Elite predictive performance (often wins Kaggle)
- ✓ Flexible to various custom loss functions
- ✓ Handles missing values. ✓ Less prone to overfitting than single models.

Disadvantages:

- ✗ Highly sensitive to hyperparameter configurations
- ✗ Sequential nature means slow training time
- ✗ Susceptible to overfitting if unconstrained
- ✗ Not ideal for very large datasets. if not tunes

5. Python Implementation

```
# Sample Dataset
X = [[2, 3], [3, 4], [1, 1], [8, 9], [9, 10], [10, 8]]
y = [0, 0, 0, 1, 1, 1]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = GradientBoostingClassifier(
    n_estimators=100, learning_rate=0.1, max_depth=3,
random_state=42)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print('Accuracy:', accuracy_score(y_test, y_pred))
```

6. Assumptions & Hyperparameters

Assumptions:

- Errors of previous models can be isolated and learned.
- Data is representative and relatively noise-free.

Key Hyperparameters:

- **n_estimators**: Number of boosting stages
- **learning_rate**: Shrinks contribution of each tree
- **max_depth**: Maximum depth of trees
- **subsample**: Fraction of samples used for training each tree
- **loss**: Loss function (e.g., 'log_loss', 'squared_error')
- **min_samples_split**: Minimum samples required to split an internal node

⚠️ WARNING / PITFALLS

⚠️ A very high learning rate or too many estimators without early stopping will guarantee extreme overfitting to noise.

💡 TIPS & TRICKS

🔑 There is an inverse relationship between learning_rate and n_estimators. If you lower learning rate, increase estimators.

❓ PRACTICE QUESTIONS

🎯 Q: How does Gradient Boosting fundamentally differ from Random Forest?

Random Forest builds independent decision trees in parallel to average predictions, while Gradient Boosting builds trees sequentially where each new model directly fits the residuals (errors) of the previous ones.

While RF strictly uses decision trees, Gradient Boosting's mathematical framework can theoretically adapt to other weak learners beyond standard trees, such as linear models, neural networks, or Naive Bayes.

🔑 **GRADIENT BOOSTING:** Turn weak learners into a grandmaster model by relentlessly attacking errors!